



南開大學
Nankai University

计算机学院
计算机网络第三次实验报告

HTTP 交互分析

姓名：林晖鹏

学号：2312966

专业：计算机科学与技术

2025 年 12 月 30 日

目录

1 实验要求	2
2 实验原理	2
2.1 Web 服务器工作原理	2
2.2 网络报文四层封装	2
2.3 Wireshark	3
3 实验流程	3
3.1 html 网站设计	3
3.2 web 服务器搭建	4
3.3 Wireshark 条件抓包	5
4 实验分析	6
4.1 HTTP 请求报文分析	6
4.1.1 数据链路层	7
4.1.2 互连层	8
4.1.3 传输层	9
4.1.4 应用层	10
4.1.5 小结	10
4.2 HTTP 响应报文分析	11
4.2.1 数据链路层	11
4.2.2 互连层	11
4.2.3 传输层	12
4.2.4 应用层	13
4.2.5 小结	13
4.3 HTTP 交互过程	13
4.3.1 设置条件过滤	13
4.3.2 并行 TCP 传输	14
4.3.3 建立 TCP 连接-三次握手	14
4.3.4 发送 HTTP 请求	15
4.3.5 服务器传输数据	15
4.3.6 发送 HTTP 响应	17
4.3.7 关闭 TCP 连接-连接释放	17
5 总结心得	18
5.1 本次实验遇到问题 & 解决	18
5.2 总结	18

1 实验要求

1. 掌握 Web 服务器的搭建方法（任选操作系统），能够制作包含指定信息（专业、学号、姓名、6 幅图像）的简易 **HTTP** 协议 Web 页面（请使用 HTTP 而不是 HTTPS）；
2. 通过浏览器获取自己编写的 Web 页面，使用 Wireshark 捕获浏览器与 Web 服务器的交互过程。
3. 分析两个报文的封装层次，包括数据链路层、互连层、传输层、应用层。
4. 对整个 HTTP 交互过程进行详细说明，使用 Wireshark 过滤器使其只显示 HTTP 协议。

2 实验原理

本实验的核心在于搭建 Web 服务器、捕获 HTTP 报文并进行分层分析。实验原理主要包括以下几个方面：

2.1 Web 服务器工作原理

Web 服务器是实现客户端与服务器端信息交互的关键组件。实验中使用的 HTTP 协议为无加密的超文本传输协议，其核心工作模型为客户端-服务器（Client-Server）模型：

1. 客户端（浏览器）向 Web 服务器发起 HTTP 请求，请求特定资源，如 HTML 页面或图像文件。
2. Web 服务器接收请求后，根据请求的 URI 查找对应资源。
3. 服务器找到对应资源后，将其封装在 HTTP 响应报文中，依托已建立的 **TCP** 可靠连接，将响应数据传输给客户端
4. 客户端解析 HTTP 响应内容并在浏览器中呈现网页。

该过程体现了请求-响应的基本交互流程，是本实验 HTTP 报文捕获分析的基础。

2.2 网络报文四层封装

HTTP 报文在传输前，会经过操作系统网络协议栈的自上而下处理，依次被封装到传输层 TCP 段、互连层 IP 数据包和数据链路层以太网帧中

1. **应用层 (HTTP)**：构成请求报文或响应报文，包括请求行/状态行、头部字段、消息体等。例如：
 - 请求报文：GET /index.html HTTP/1.1
 - 响应报文：HTTP/1.1 200 OK
2. **传输层 (TCP)**：提供可靠数据传输，负责将应用层报文分段，并添加源/目的端口号（如 HTTP 默认端口 80），保证报文顺序和完整性。
3. **互连层 (IP)**：在网络中进行寻址与路由，将 TCP 段封装为 IP 数据包，并附加源/目的 IP 地址。
4. **数据链路层 (以太网)**：在同一局域网内传输数据，将 IP 数据包封装为以太网帧，包含源/目的 MAC 地址以及帧检验序列（FCS）。

当报文到达目标主机后，会按照自下而上的顺序逐层解封装，剥离各层首部信息，最终将应用层 HTTP 报文交付给浏览器应用程序。

2.3 Wireshark

Wireshark 是一款网络协议分析工具，能够在数据链路层捕获网络报文并解析各层协议：

- Wireshark 利用网卡的混杂模式捕获经过网卡的数据帧。
- 捕获的每个数据帧会被逐层解析，显示其数据链路层、互连层、传输层和应用层的内容。
- 用户可以使用**协议过滤器**筛选目标报文，例如只显示 HTTP 协议报文，从而分析客户端与 Web 服务器的交互过程。

通过这个软件可以对 Web 服务器交互过程进行抓包，从而分析 HTTP 交互过程

3 实验流程

3.1 html 网站设计

由于这个 html 网页并不是本实验的重点，这里直接放源代码和示意图，不做赘述：

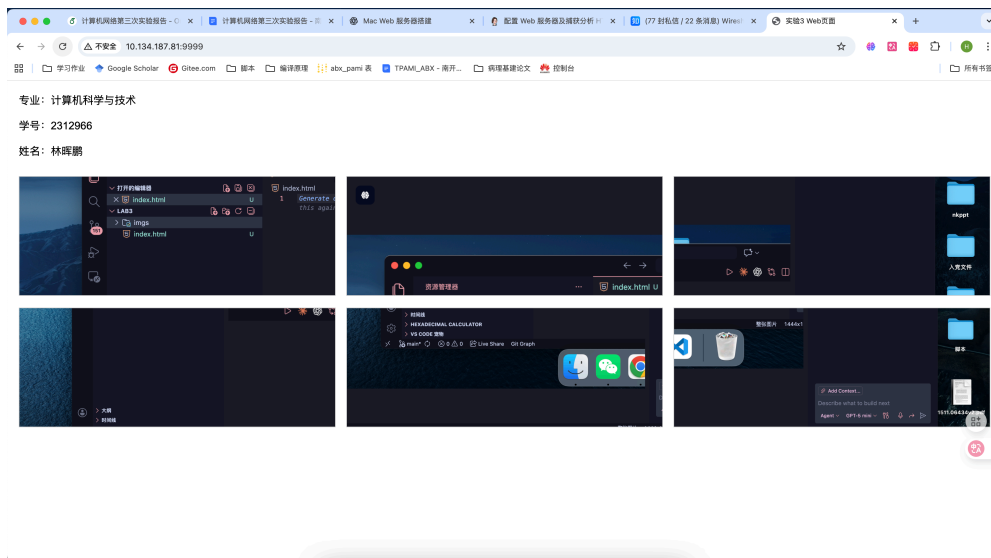


图 3.1: 网站展示

```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
3
4 <head>
5   <meta charset="UTF-8">
6   <title>实验3 Web页面</title>
7   <style>
8     /* 简单样式，确保页面简洁，满足实验要求即可 */
9     body {
10       margin: 20px;
11       font-family: Arial, sans-serif;
12     }
13
```

```
14 .info {
15     font-size: 18px;
16     margin-bottom: 30px;
17 }
18
19 .images {
20     display: grid;
21     grid-template-columns: repeat(3, 1fr);
22     gap: 20px;
23 }
24 .images img {
25     width: 100%;
26     height: 200px;
27     object-fit: cover;
28     border: 1px solid #ccc;
29 }
30 </style>
31 </head>
32
33 <body>
34     <!-- 核心信息：专业、学号、姓名 -->
35     <div class="info">
36         <p>专业：计算机科学与技术</p>
37         <p>学号：2312966</p>
38         <p>姓名：林晖鹏</p>
39     </div>
40
41     <!-- 6幅图像展示 -->
42     <div class="images">
43         
44         
45         
46         
47         
48         
49     </div>
50 </body>
51
52 </html>
```

3.2 web 服务器搭建

我们这里先尝试了两种服务器搭建方式

1. python 启动服务器

输入下面这个指令在终端，进行服务器运行，然后直接访问网站 <http://localhost:9999>.

```
1 python3 -m http.server 9999
```

2. Apache 服务器搭建

首先启动 Apache，终端输入指令：

```
1 sudo apachectl start
```

然后将 index.html 以及相关资源文件放在 Apache 的默认网站目录下： */Library/WebServer/Documents*。

为了想不回环，这里还尝试修改 config 配置文件，将部署地址切换为另一个网卡上面，终端输入下面指令，然后将 *Listen 80* 改为 *Listen < 你的本地 ip 地址 >:80*

```
1 sudo nano /etc/apache2/httpd.conf
2 # 再重启Apache
3 sudo apachectl restart
```

但是 在后面分析数据链路层的时候，发现如果是本地回环的话（本地 ip 服务器搭建，本地浏览器获取），由于本地回环通信不经过真实物理网卡和局域网链路，导致会出现操作系统对数据链路层进行优化，跳过了标准以太网帧的封装流程。在 Wireshark 捕获的报文中，链路层未显示完整的以太网帧结构（无源 MAC 地址、目的 MAC 地址、类型字段 0x0800 等），仅出现简化的占位符数据 02 00 00 00。如下图所示：

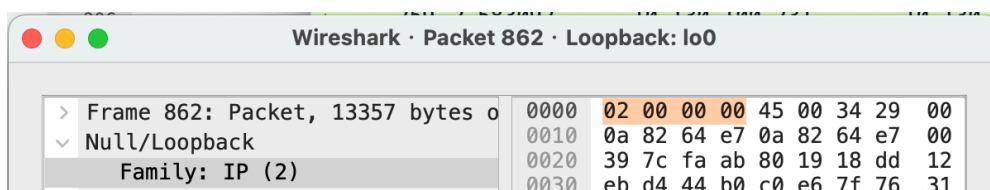


图 3.2: 数据链路层信息被优化

所以这里在同学的电脑上面用 python 配置 Web 服务器，然后两个电脑在同一局域网下，通过访问同学电脑的 ip 地址进行浏览器交互，抓到的 http 包终于有数据链路层信息了！

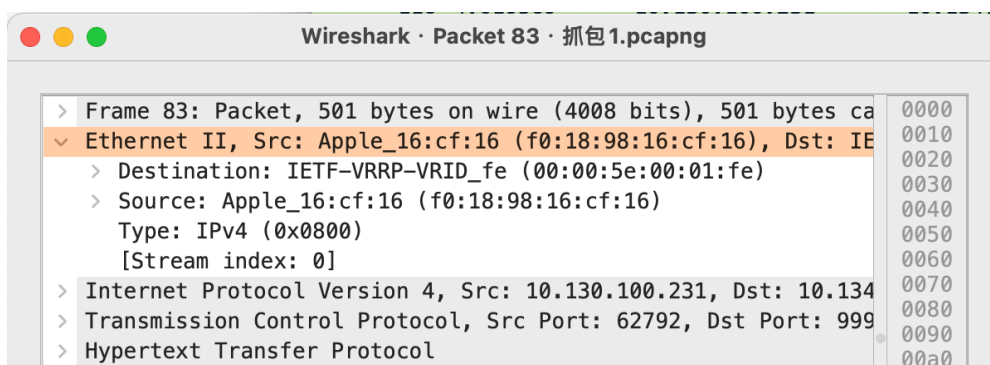
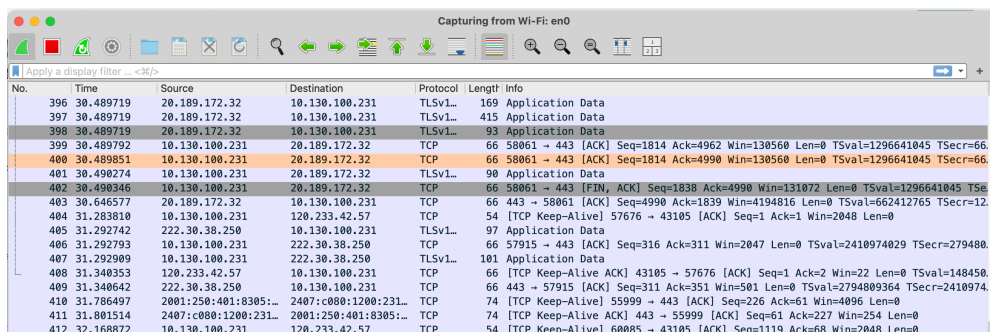


图 3.3: 数据链路层未被优化

3.3 Wireshark 条件抓包

在浏览器链接服务器网站前，启动 Wireshark 进行抓捕信息，点击左上角小鲨鱼旗。

但是如下图可以看到，有很多后台前台应用以及操作系统会建立多个网络连接。导致我们想分析特定网络交互的话十分困难。



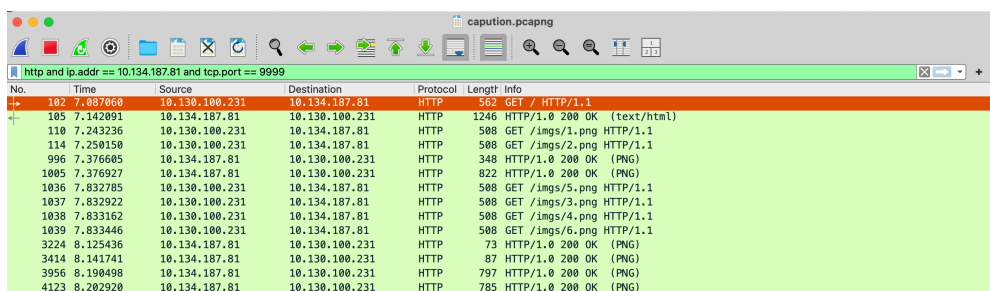
No.	Time	Source	Destination	Protocol	Length	Info
396	30.489719	20.189.172.32	10.130.100.231	TLSv1...	169	Application Data
397	30.489719	20.189.172.32	10.130.100.231	TLSv1...	415	Application Data
398	30.489719	20.189.172.32	10.130.100.231	TLSv1...	93	Application Data
399	30.489792	10.130.100.231	20.189.172.32	TCP	66	58061 → 443 [ACK] Seq=1814 Ack=4962 Win=130560 Len=0 TSval=1296641045 TSecr=66
400	30.489851	10.130.100.231	20.189.172.32	TCP	66	58061 → 443 [ACK] Seq=1814 Ack=4990 Win=130560 Len=0 TSval=1296641045 TSecr=66
401	30.490274	10.130.100.231	20.189.172.32	TLSv1...	90	Application Data
402	30.490346	10.130.100.231	20.189.172.32	TCP	66	58061 → 443 [FIN, ACK] Seq=1838 Ack=4990 Win=131072 Len=0 TSval=1296641045 TSecr=66
403	30.646577	20.189.172.32	10.130.100.231	TCP	66	443 → 58061 [ACK] Seq=4990 Ack=1839 Win=4194816 Len=0 TSval=662412765 TSecr=12
404	31.283810	10.130.100.231	120.233.42.57	TCP	54	[TCP Keep-Alive] 57676 → 43105 [ACK] Seq=1 Ack=1 Win=2048 Len=0
405	31.292742	222.30.38.250	10.130.100.231	TLSv1...	97	Application Data
406	31.292793	10.130.100.231	222.30.38.250	TCP	66	57915 → 443 [ACK] Seq=316 Ack=311 Win=2047 Len=0 TSval=2410974029 TSecr=279480
407	31.292909	10.130.100.231	222.30.38.250	TLSv1...	101	Application Data
408	31.340353	120.233.42.57	10.130.100.231	TCP	66	[TCP Keep-Alive ACK] 43105 → 57676 [ACK] Seq=1 Ack=2 Win=22 Len=0 TSval=148450
409	31.340642	222.30.38.250	10.130.100.231	TCP	66	443 → 57915 [ACK] Seq=311 Ack=351 Win=501 Len=0 TSval=2794809364 TSecr=2410974
410	31.786497	2001:250:401:8305::...	2407:c080:1200:231::...	TCP	74	[TCP Keep-Alive] 55999 → 443 [ACK] Seq=226 Ack=61 Win=4096 Len=0
411	31.801514	2407:c080:1200:231::...	2001:250:401:8305::...	TCP	74	[TCP Keep-Alive ACK] 443 → 55999 [ACK] Seq=61 Ack=227 Win=254 Len=0
412	32.168872	10.130.100.231	120.233.42.57	TCP	54	[TCP Keep-Alive] 60085 → 43105 [ACK] Seq=1119 Ack=68 Win=2048 Len=0

图 3.4: 无条件抓捕

因此，在进行报文分析时，需要对捕获的数据进行筛选。为仅显示浏览器与 Web 服务器之间的 HTTP 交互过程，在 Wireshark 的显示过滤栏中设置如下过滤条件：

`http and ip.addr == < 目标 IP 地址 > and tcp.port == 9999`

可以看到，在开启显示过滤条件后，Wireshark 抓包界面中仅保留了满足条件的 HTTP 报文，如图 4.19 所示。此时显示的报文均为浏览器与目标 Web 服务器之间、基于 TCP 端口 9999 的 HTTP 请求与响应过程，其余无关协议和主机通信已被有效过滤。



No.	Time	Source	Destination	Protocol	Length	Info
102	7.007060	10.130.100.231	10.134.187.81	HTTP	562	GET / HTTP/1.1
105	7.142091	10.134.187.81	10.130.100.231	HTTP	1246	HTTP/1.0 200 OK (text/html)
110	7.243236	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/1.png HTTP/1.1
114	7.250150	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/2.png HTTP/1.1
996	7.376085	10.134.187.81	10.130.100.231	HTTP	348	HTTP/1.0 200 OK (PNG)
1005	7.376927	10.134.187.81	10.130.100.231	HTTP	822	HTTP/1.0 200 OK (PNG)
1036	7.832785	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/5.png HTTP/1.1
1037	7.832922	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/3.png HTTP/1.1
1038	7.833162	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/4.png HTTP/1.1
1039	7.833446	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/6.png HTTP/1.1
3224	8.125436	10.134.187.81	10.130.100.231	HTTP	73	HTTP/1.0 200 OK (PNG)
3414	8.141741	10.134.187.81	10.130.100.231	HTTP	87	HTTP/1.0 200 OK (PNG)
3956	8.190498	10.134.187.81	10.130.100.231	HTTP	797	HTTP/1.0 200 OK (PNG)
4123	8.202928	10.134.187.81	10.130.100.231	HTTP	785	HTTP/1.0 200 OK (PNG)

图 3.5: 开启条件过滤后

注意，这里为了使用浏览器缓存，需要按 F12 打开网络界面关闭‘启用缓存’。不然如果之前打开过，那么 HTTP 的响应报文是缓存响应。

完成抓包之后，下面我们对抓捕的信息进行分析。

4 实验分析

下面我们依次先对两个 HTTP 报文进行分析，然后在详解 HTTP 的交互流程。

4.1 HTTP 请求报文分析

选取浏览器向 Web 服务器请求网页资源时发送的一条 HTTP GET 报文，对其封装层次进行分析。

4.1.1 数据链路层

在数据链路层，该报文被封装为以太网帧。以太网帧中包含源 MAC 地址和目的 MAC 地址，其中源 MAC 地址为客户端主机网卡地址，目的 MAC 地址为服务器主机或中间网络设备的地址，用于在局域网内完成数据传输。



图 4.6: 数据链路层

同上面分析一致，从数据链路层的信息中我们可以读出：

1. 帧类型

该数据帧的类型为 **Ethernet II (以太网第二版帧)**。Ethernet II 是目前最常用的以太网帧格式，其通过帧头中的类型字段 (Type) 直接指明上层所承载的网络层协议类型，为后续协议解析提供依据。

2. MAC 地址信息

在该以太网帧中，目的 MAC 地址为 *IETF-VRRP-VRID_fe*，其十六进制表示为 **00:00:5e:00:01:fe**；源 MAC 地址为 **Apple_16:cf:16**，其十六进制表示为 **f0:18:98:16:cf:16**。

其中，源 MAC 地址对应本地主机的物理网卡地址，而目的 MAC 地址为通信过程中对端设备或下一跳网络设备的 MAC 地址，用于在局域网内完成帧级别的数据转发。

3. MAC 地址属性分析

对 MAC 地址的二进制最低有效位进行分析，可判断其地址属性：

- 目的 MAC 地址属性：
 - LG 位 = 0，表示该地址为全局唯一地址（由厂商分配，非本地管理地址）；
 - IG 位 = 0，表示该地址为单播地址，用于点对点通信，而非广播或组播。
- 源 MAC 地址属性：
 - LG 位 = 0，表示全局唯一地址；
 - IG 位 = 0，表示单播地址。

上面分析表明，该以太网帧属于典型的**单播通信帧**，数据在局域网内由发送端直接发送至目标设备。

4. 协议类型

以太网帧头中的类型字段为 **0x0800**，该字段表明其上层承载的协议为 **IPv4**。因此，该以太网帧的载荷部分为 IPv4 数据包。

5. 帧校验序列 (FCS)

按照以太网帧标准，帧尾应包含帧校验序列 (Frame Check Sequence, FCS)，用于接收端进行差错检测。然而，在本实验环境下使用的 macOS 操作系统中，网卡驱动通常在数据上送至操作系统协议栈之前即完成 FCS 校验并将其剥离，因此 Wireshark 无法捕获并显示该字段。很遗憾。

4.1.2 互连层

在互连层，该以太网帧的载荷部分为 IP 数据包。IP 头部中包含源 IP 地址和目的 IP 地址，分别对应客户端主机 IP 和 Web 服务器 IP（如 10.130.100.231），用于实现网络层的寻址与路由转发。

```
Transmission Control Protocol, Src Port: 59309, Dst Port: 9999, Seq: 1, Ack: 1, Len: 496
  Source Port: 59309
  Destination Port: 9999
  [Stream index: 15]
  [Stream Packet Number: 4]
  > [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 496]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 895580893
  [Next Sequence Number: 497 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1756807731
  1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x018 (PSH, ACK)
  Window: 2058
  [Calculated window size: 131712]
  [Window size scaling factor: 64]
  Checksum: 0xdec7 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  [Timestamps]
  > [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]
  TCP payload (496 bytes)
```

图 4.7: 互连层 (IPv4)

在数据链路层完成以太网帧的解析后，进一步对其载荷部分进行分析，可以得到该报文在互连层中的 IPv4 协议封装信息。根据 Wireshark 的解析结果 4.7，IPv4 头部各字段含义如下：

1. IP 版本与头部长度

该数据包的 IP 版本号为 4，表明其采用 **IPv4** 协议。IP 头部长度为 20 字节（5 个 32 位字），说明该报文未包含任何 IPv4 选项字段，使用的是标准 IPv4 头部格式。

2. 总长度字段

IP 数据包的总长度为 548 字节，该长度包含 IPv4 头部以及其所承载的上层数据载荷，反映了该报文在网络层的整体大小。

3. 服务质量字段 (DSCP / ECN)

DSCP 值为 0，表示该数据包采用默认的“**Best Effort**”服务模式；**ECN** 字段为 Not-ECT，表明该数据包不支持显式拥塞通知机制，未参与网络拥塞反馈控制。

4. 分片控制字段

该数据包的标识字段为 0x0000，其中“**Don't Fragment**”标志位被设置，表示该数据包在传输过程中不允许被分片；“**More Fragments**”标志位未设置，分片偏移为 0，说明该数据包为唯一分片。这种设置有助于保证上层协议数据的完整性。

5. 生存时间与上层协议指示

数据包的生存时间（**TTL**）为 64，该取值是 Linux、Unix 及 macOS 系统中常见的默认初始值（本次实验客户端使用电脑为 mac），说明该数据包尚未经过较多路由转发。协议字段的值为 6，表明其上层封装的协议为 TCP。

6. 源 IP 与目的 IP 地址

源 IP 地址为 10.130.100.231，目的 IP 地址为 10.134.187.81，二者均属于私有地址空间，表明该通信发生在内网环境（这里通过校园网连接同学的电脑搭建的服务器）中，用于客户端与 Web 服务器之间的数据交互。

4.1.3 传输层

在传输层，IP 数据包的载荷为 TCP 报文段。TCP 头部中包含源端口号和目的端口号，其中目的端口号为 Web 服务器监听的端口（如 9999）。TCP 协议通过序列号和确认号机制，保证 HTTP 请求数据的可靠传输。

No.	Time	Source	Destination	Protocol	Length	Info
7.075797	10.130.100.231	10.134.187.81	TCP	78	59310 → 9999 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3518264356 TSecr=SACK_PERM	
7.086759	10.134.187.81	10.130.100.231	TCP	74	9999 → 59310 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=15527052 TSecr=3518264356	
7.086878	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=3518264367 TSecr=15527052	
7.263381	10.134.187.81	10.130.100.231	TCP	255	9999 → 59310 [PSH, ACK] Seq=1 Ack=443 Win=65824 Len=189 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.263450	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=190 Win=131520 Len=0 TSval=3518264544 TSecr=15527227	
7.268079	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=190 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=1638 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=3806 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.268935	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=534 Win=127232 Len=0 TSval=3518264549 TSecr=15527227	
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=4534 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=5982 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.269006	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=7430 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.269020	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=878 Win=122808 Len=0 TSval=3518264549 TSecr=15527227	
7.269044	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=878 Win=131072 Len=0 TSval=3518264549 TSecr=15527227	
7.269819	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=8878 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=18326 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=11774 Ack=443 Win=59024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]	
7.269875	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=13222 Win=126720 Len=0 TSval=3518264549 TSecr=15527227	
7.269952	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=13222 Win=131072 Len=0 TSval=3518264550 TSecr=15527227	
7.275377	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=13222 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275378	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=14670 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275379	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=16118 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=17566 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=19014 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275478	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=20462 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275479	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=21910 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	
7.275484	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=28602 Win=127776 Len=0 TSval=3518264556 TSecr=15527240	
7.275509	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=23358 Win=120896 Len=0 TSval=3518264556 TSecr=15527240	
7.275544	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=23358 Ack=443 Win=59024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]	

图 4.8: 传输层

在互连层确认该 IPv4 数据包承载的是 TCP 协议后，进一步对传输层的 TCP 报文段结构进行分析，其解析结果 4.8 如下：

1. 端口号信息

TCP 报文段的源端口号为 59309，目的端口号为 9999。其中，端口 9999 为非标准 HTTP 端口，由于防止监听错误，这里实验中搭建的 Web 服务器采用了自定义监听端口。

2. 序列号与确认号

报文中的 Seq 序列号和 Ack 确认号在 Wireshark 中以相对值形式显示，均为 1，对应的原始序列号分别为 895580893 和 1756807731。该字段用于保证 TCP 数据传输的有序性与可靠性。

3. TCP 头部长度与选项

TCP 头部长度为 32 字节，相比标准 20 字节头部多出 12 字节，说明该报文包含 TCP 选项字段。其中包含时间戳选项，用于 RTT 测量以及防止序列号回绕。

4. 标志位

该 TCP 报文段设置了 PSH 与 ACK 标志位。其中，ACK 标志表示对已接收数据的确认；PSH 标志表示发送端请求接收端在收到数据后立即将其递交给应用层，常见于 HTTP 请求等交互式通信场景。

5. 窗口大小与流量控制

TCP 窗口大小字段的原始值为 2058，结合窗口缩放因子 64，其实际接收窗口大小为 131712 字节，表明该连接支持较大的接收窗口，有利于提高数据传输效率。

6. 数据载荷长度

该 TCP 报文段的载荷 **payload** 长度为 496 字节，用于承载上层的 HTTP 应用层数据。

4.1.4 应用层

在传输层完成 TCP 报文段的解析后，可以进一步对其承载的应用层数据进行分析。根据 Wireshark 对应用层内容的解析结果，该报文为一条 HTTP GET 请求，其具体信息如下：



```

Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
  Host: 10.134.187.81:9999\r\n
  Connection: keep-alive\r\n
  Pragma: no-cache\r\n
  Cache-Control: no-cache\r\n
  Upgrade-Insecure-Requests: 1\r\n
  User-Agent: Mozilla/5.0 (Linux; Android 6.0; Nexus 5 Build/MRA58N) AppleWebKit/537.36 (KHTML, like Gecko) ...
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,a...
  Accept-Encoding: gzip, deflate\r\n
  Accept-Language: zh-CN,zh;q=0.9\r\n
  \r\n
  [Response in frame: 105]
  [Full request URI: http://10.134.187.81:9999/]

```

图 4.9: 应用层

1. 请求方法与资源路径

HTTP 请求行内容为 **GET / HTTP/1.1**，表明客户端使用 GET 方法请求服务器根目录下的资源，协议版本为 **HTTP/1.1**，与实验要求里面使用 HTTP 相符。

2. 目标服务器标识

请求头中的 Host 字段为 **10.134.187.81:9999**，明确指定了目标服务器的 IP 地址及其监听端口号，与传输层目的端口信息一致。

3. 用户代理信息

User-Agent 字段标识客户端为 Android Nexus 5 设备的浏览器环境，该字段用于向服务器说明客户端的软件与设备类型，便于服务器进行兼容性处理。

4. 内容协商相关字段

Accept 字段表明客户端可接受的响应内容类型，**Accept-Encoding** 字段表明客户端支持 **gzip** 和 **deflate** 等压缩方式，有助于提升数据传输效率。

5. 连接管理方式

Connection 字段被设置为 **keep-alive**，表示客户端希望在当前请求完成后保持 TCP 连接不断开，以便后续继续发送 HTTP 请求，减少重复建立连接带来的开销。

4.1.5 小结

本次捕获并分析的 HTTP 请求报文完整体现了浏览器向 Web 服务器获取网页资源的典型通信过程。从底层的数据链路层以太网帧开始，到互连层 IPv4 数据包、传输层 TCP 报文段，再到最上层的 HTTP GET 请求，各层协议分工明确、逐级封装，共同完成了一次可靠的应用层请求传输。

其中，以太网帧负责在局域网内完成帧级转发；IPv4 协议实现了端到端的逻辑寻址与路由；TCP 协议通过序列号、确认号及窗口机制保障了数据传输的可靠性和有序性；而 HTTP 协议则明确表达了客户端对服务器资源的访问意图。该请求报文成功发送后，服务器将依据请求内容生成相应的 HTTP 响应报文，并沿相同的协议栈路径返回给客户端。

因此，在完成对 HTTP 请求报文的封装与内容分析后，这里下一步将对该请求所对应的 HTTP 响应报文进行分析。

4.2 HTTP 响应报文分析

响应报文由 Web 服务器在接收到客户端的 GET 请求后生成，并沿与请求报文相反的方向返回客户端。由于请求与响应在协议封装层次上保持一致，以下分析将只重点关注响应方向各层字段的变化及其所反映的通信特征，并结合请求报文进行对比说明，重复的概念就不过多解释了。

4.2.1 数据链路层

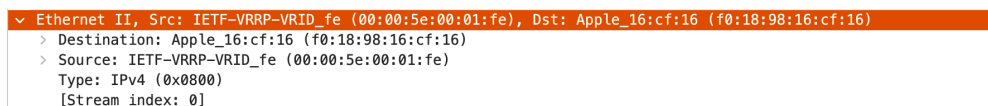


图 4.10: 响应报文数据链路层

在数据链路层，响应报文同样被封装为 Ethernet II 类型的以太网帧，其帧格式与请求报文保持一致。与请求方向不同的是，该响应帧的源 MAC 地址与目的 MAC 地址发生了对调。

具体而言，源 MAC 地址为 *IETF-VRRP-VRID_fe* (00:00:5e:00:01:fe)，目的 MAC 地址为客户端主机的网卡地址 **Apple_16:cf:16** (f0:18:98:16:cf:16)。这表明响应数据经由 VRRP 虚拟网关转发，并最终送达客户端设备，符合局域网内单播通信的特征。

4.2.2 互连层

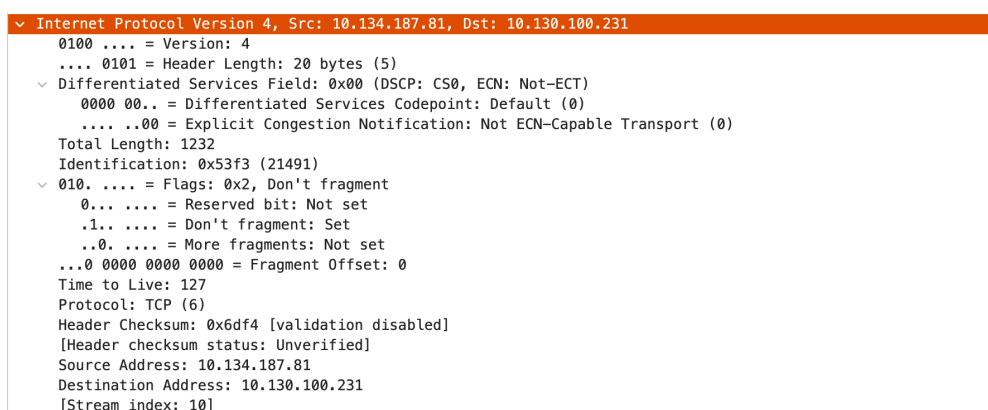


图 4.11: 响应报文互连层

在互连层，响应报文的载荷仍为 IPv4 数据包，其头部结构与请求报文一致，但关键字段取值体现了通信方向的反转。

响应数据包的源 IP 地址为 10.134.187.81, 对应 Web 服务器主机; 目的 IP 地址为 10.130.100.231, 即客户端主机地址, 与请求报文中的源、目的 IP 地址正好相反。该数据包的总长度为 1232 字节, 明显大于请求报文的 548 字节, 说明服务器在响应中返回了较多的应用层数据 (这里是 html 文件)。

此外, 响应报文的 TTL 值为 127, 与请求报文中 TTL=64 形成对比。这一特征通常与服务器操作系统的默认初始 TTL 值有关, 结合后续应用层信息, 可以推测服务器端运行环境可能为 Windows 系统。IPv4 头部同样设置了 “Don’t Fragment” 标志位, 表明该响应数据包在传输过程中不允许被分片。

4.2.3 传输层

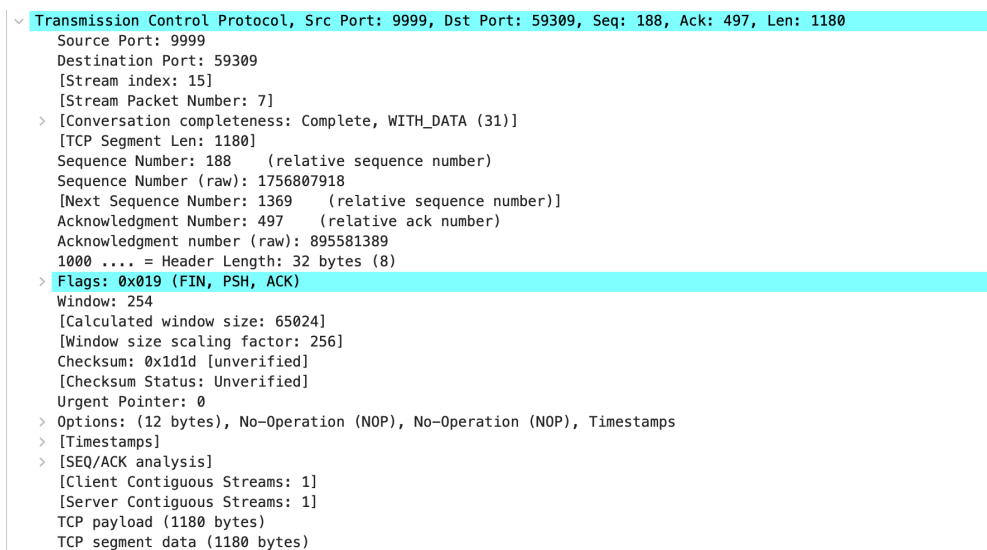


图 4.12: 响应报文传输层

在传输层, 响应报文承载的是 TCP 报文段, 其端口号信息与请求报文呈现明显的对称关系: 源端口号为服务器监听端口 9999, 目的端口号为客户端的临时端口 59309。

从序列号与确认号来看, 响应报文中的确认号为 497, 恰好对应请求报文中已发送的 HTTP 请求数据长度, 说明服务器已正确接收并确认客户端的请求内容; 同时, 响应报文的序列号从 188 开始递增, 用于标识服务器向客户端发送的应用层数据流。

值得注意的是, 该 TCP 报文段同时设置了 **FIN**、**PSH** 和 **ACK** 标志位。其中, PSH 标志表示服务器在发送完数据后立即将其推送至客户端应用层; FIN 标志则表明服务器在完成数据发送后主动关闭连接。这一点与请求报文中使用的 **Connection: keep-alive** 形成对比, 反映出服务器端并未维持持久连接, 因为这里服务器通信通常是**并行多 TCP 传输**, 每个 TCP 都只传输一个文件, 所以传完之后就关闭这个 TCP 传输了。

结合 TCP 窗口大小及缩放因子可以看出, 该连接仍支持较大的接收窗口, 但由于服务器在发送响应后立即释放连接, 整体通信过程以一次完整的请求—响应交互结束。

4.2.4 应用层



```
▼ Hypertext Transfer Protocol
  ▼ HTTP/1.0 200 OK\r\n
    Response Version: HTTP/1.0
    Status Code: 200
    [Status Code Description: OK]
    Response Phrase: OK
    Server: SimpleHTTP/0.6 Python/3.13.5\r\n
    Date: Mon, 29 Dec 2025 14:52:14 GMT\r\n
    Content-type: text/html\r\n
    > Content-Length: 1180\r\n
    Last-Modified: Mon, 29 Dec 2025 12:05:21 GMT\r\n
    \r\n
    [Request in frame: 102]
    [Time since request: 55.031000 milliseconds]
    [Request URI: /]
    [Full request URI: http://10.134.187.81:9999/]
    File Data: 1180 bytes
```

图 4.13: 响应报文应用层

在应用层，响应报文为一条标准的 HTTP 响应消息，其协议版本为 **HTTP/1.0**，状态行为 200 OK，表明服务器已成功处理客户端的请求并返回所需资源。

与请求中使用的 HTTP/1.1 不同，服务器响应采用 HTTP/1.0 协议版本，这也是其在发送完数据后立即关闭 TCP 连接的重要原因之一。HTTP/1.0 默认不支持持久连接，除非显式指定 **Connection: keep-alive**。

响应头部中包含 **Content-Type: text/html** 与 **Content-Length: 1180** 字段，明确指示响应实体为 HTML 文本内容，且实体主体长度为 1180 字节。该长度与 TCP 层中实际承载的应用层数据大小一致，验证了响应数据的完整性与一致性。

此外，**Server** 字段标识服务器为 **SimpleHTTP/0.6 Python/3.13.5**，表明该 Web 服务由 Python 内置的简易 HTTP 服务器提供，主要用于静态文件访问。结合响应时间约 55 毫秒可以看出，在内网环境下，该服务器对静态资源请求的处理与响应效率较高。

4.2.5 小结

HTTP 响应报文与请求报文共同构成了一次完整、规范的 Web 通信过程，清晰体现了客户端请求资源、服务器返回结果并主动关闭连接的典型 HTTP 交互模式。

4.3 HTTP 交互过程

接下来，将按照 **建立 TCP 连接 → 发送 HTTP 请求 → 服务器传输数据 -> 发送 HTTP 响应 → 关闭 TCP 连接** 的顺序，对浏览器与 Web 服务器之间的完整 HTTP 交互过程进行详细分析。

4.3.1 设置条件过滤

由于这里 HTTP 只是应用层协议，本身不负责连接的建立与释放，而连接建立 / 可靠传输 / 关闭连接全部由 TCP 完成，所以这里我们要把过滤条件设置为

```
(http or tcp) and ip.addr == < 目标 IP 地址 > and tcp.port == 9999
```

在该过滤条件下，抓包结果仅保留与目标 Web 服务器相关的 TCP 与 HTTP 报文，既能够观察 TCP 连接的建立与释放过程，又可以分析 HTTP 请求与响应数据在 TCP 连接中的传输情况。

4.3.2 并行 TCP 传输

在对抓包结果进行分析时，如图 4.14，可以观察到客户端几乎同时建立了多个指向同一服务器端口（9999）的 TCP 连接，其源端口号分别为 59309 和 59310，对应不同的 TCP 会话。这一现象表明，在本次 Web 访问过程中，浏览器采用了并行 TCP 连接的方式进行数据传输。

这个的主要原因在于浏览器在获取 HTML 页面内容后，还需要继续请求页面中引用的图片等静态资源。为了减少资源下载过程中的等待时间、提升页面加载效率，浏览器通常会并行建立多个 TCP 连接，从而实现对不同资源的并发请求与传输。

结合本实验环境可以进一步分析发现，服务器返回的 HTTP 响应采用的是 HTTP/1.0 协议版本。而由于 HTTP/1.0 默认不支持持久连接，服务器在完成一次请求响应后即主动关闭 TCP 连接，导致浏览器在请求后续资源时需要重新建立新的 TCP 连接。在此情况下，浏览器通过并行建立多个 TCP 连接，以避免资源请求串行化带来的性能下降。

这里抓包中出现的多个并行 TCP 三次握手过程属于浏览器在加载多资源网页时的正常行为，体现了现代浏览器通过并行 TCP 传输机制优化 Web 访问性能的设计思想。

7.073981	10.130.100.231	10.134.187.81	TCP	78	59309 → 9999	[SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=830634450 TSecr=0 SA...
7.075797	10.130.100.231	10.134.187.81	TCP	78	59310 → 9999	[SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3518264356 TSecr=0 S...
7.086758	10.134.187.81	10.130.100.231	TCP	74	9999 → 59309	[ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSva...
7.086879	10.134.187.81	10.130.100.231	TCP	74	9999 → 59310	[ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSva...

图 4.14: 并行 TCP 连接握手观察

所以下面我们只观察一个客户端端口，只对某个图片传输的 HTTP 交互过程进行分析，过滤条件转换为：

(http or tcp) and ip.addr == < 目标 IP 地址 > and tcp.port == 9999 and tcp.port == 59310

4.3.3 建立 TCP 连接-三次握手

在访问 Web 服务器之前，客户端必须先与服务器建立一个可靠的 TCP 连接。根据抓包分析结果，如图 4.15 所示，TCP 连接的三次握手过程具体如下：

7.075797	10.130.100.231	10.134.187.81	TCP	78	59310 → 9999	[SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3518264356 TSecr=0 SACK_PERM
7.086759	10.134.187.81	10.130.100.231	TCP	74	9999 → 59310	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=15527052 TSecr=3518264356
7.086878	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=3518264367 TSecr=15527052
7.263381	10.134.187.81	10.130.100.231	TCP	255	9999 → 59310	[PSH, ACK] Seq=1 Ack=443 Win=65024 Len=189 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.263450	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=443 Ack=190 Win=131520 Len=0 TSval=3518264544 TSecr=15527227
7.268879	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=190 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=1638 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=3086 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268935	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=443 Ack=4534 Win=127232 Len=0 TSval=3518264549 TSecr=15527227
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=4534 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=5982 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269006	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=7430 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269028	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=443 Ack=8878 Win=122880 Len=0 TSval=3518264549 TSecr=15527227
7.269044	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999	[ACK] Seq=443 Ack=8878 Win=131072 Len=0 TSval=3518264549 TSecr=15527227
7.269819	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=8878 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=10326 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=11774 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269875	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=443 Ack=13222 Win=126728 Len=0 TSval=3518264549 TSecr=15527227
7.269952	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999	[ACK] Seq=443 Ack=13222 Win=131072 Len=0 TSval=3518264550 TSecr=15527227
7.275377	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=13222 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275378	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=14670 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275379	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=16118 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=17566 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=19014 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275478	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=20462 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275479	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=21910 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275484	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=443 Ack=20462 Win=123776 Len=0 TSval=3518264556 TSecr=15527240
7.275509	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999	[ACK] Seq=443 Ack=23358 Win=120896 Len=0 TSval=3518264556 TSecr=15527240
7.275544	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310	[ACK] Seq=23358 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]

图 4.15: 三次握手

1. 客户端发送 SYN（初始化连接）客户端向服务器发送 SYN 报文，初始化序列号 Seq=0，并告知服务器其接收窗口大小及支持的 TCP 选项（MSS、时间戳、窗口缩放、SACK）。该报文用于

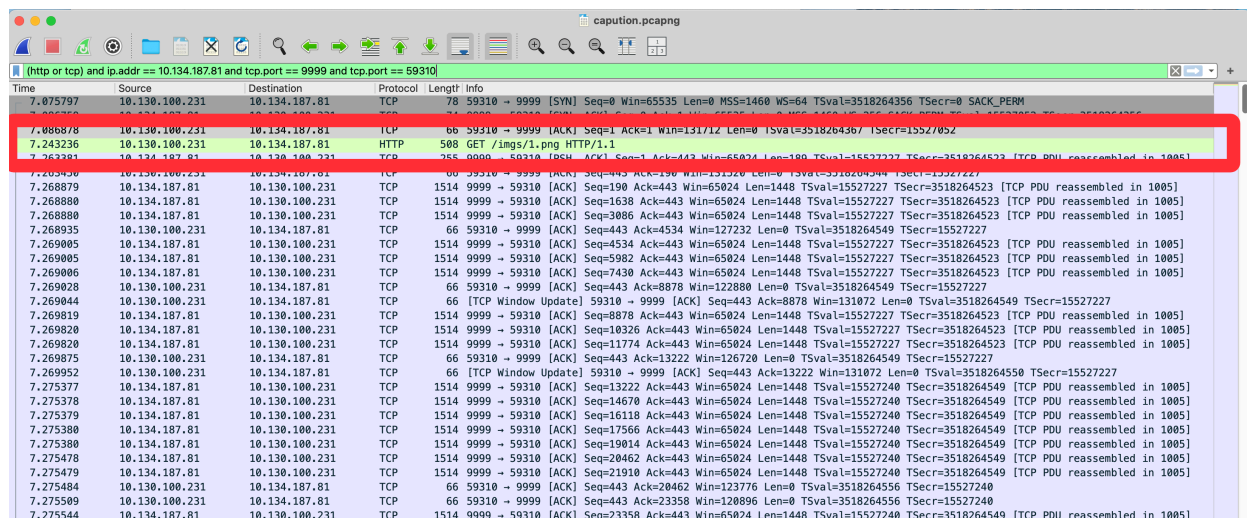
启动 TCP 连接，并交换双方参数。

2. **服务器返回 SYN-ACK (确认连接)** 服务器回应客户端的 SYN，同时发送自己的初始序列号 Seq=0，并确认客户端的序列号 Ack=1。服务器同样提供接收窗口信息及 TCP 选项 (MSS、窗口缩放、SACK、时间戳)，完成双向参数交换。
3. **客户端发送 ACK (完成握手)** 客户端确认服务器序列号 Ack=1，最终确认双方序列号和窗口信息。此时 TCP 连接建立完成，为后续 HTTP 请求和响应提供可靠传输通道。

如图所示，客户端和服务端均启用了 MSS、窗口缩放、时间戳和 SACK 等 TCP 性能优化机制，可提高内网传输效率。整个握手过程中数据包较小，主要用于交换序列号与确认号，并协商窗口和选项。

4.3.4 发送 HTTP 请求

在握手成功之后，客户端向服务器发送 HTTP 请求，HTTP 包的内容在前文已经讲过，这里不再赘述



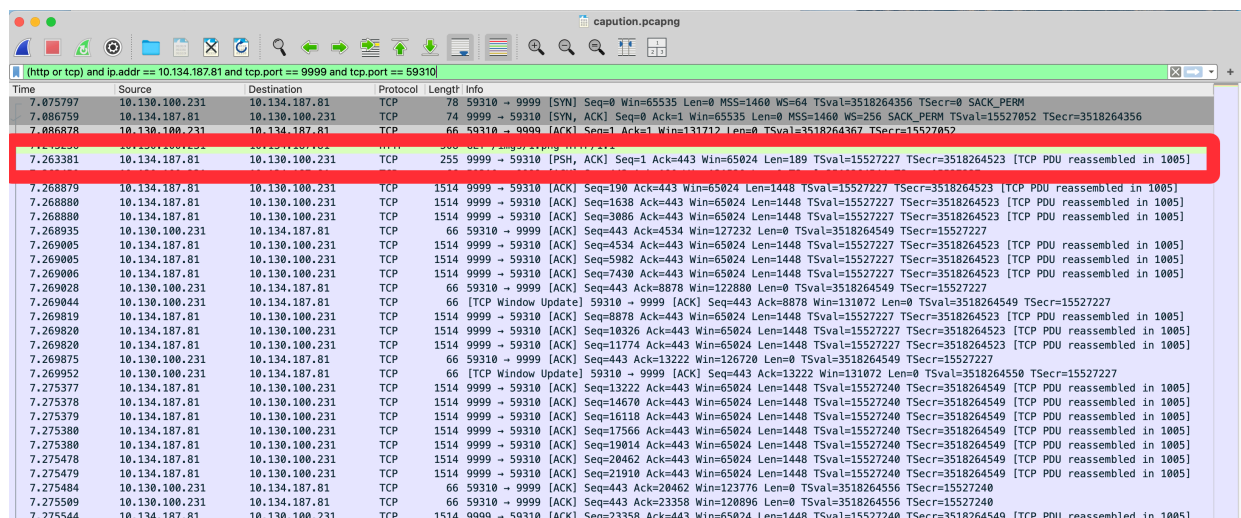
Time	Source	Destination	Protocol	Length	Info
7.075797	10.130.100.231	10.134.187.81	TCP	78	59310 → 9999 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3518264356 TSecr=0 SACK_PERM
7.080878	10.130.100.231	10.134.187.81	TCP	60	59310 → 9999 [ACK] Seq=1 Ack=1 Win=131/12 Len=0 TSval=3518264367 TSecr=15527052
7.243236	10.130.100.231	10.134.187.81	HTTP	508	GET /img5/1.png HTTP/1.1
7.263281	10.134.187.81	10.130.100.231	TCP	366	9999 → 59310 [RST] Seq=1 Ack=443 Win=65024 Len=0 TSval=155272227 TSecr=3518264523 [TCP RST reassembled in 1005]
7.268879	10.134.187.81	10.130.100.231	TCP	60	59310 → 9999 [ACK] Seq=190 Ack=443 Win=65024 Len=0 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=1638 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268935	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=4534 Win=127232 Len=0 TSval=3518264549 TSecr=155272227
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=4534 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=5982 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269006	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=7430 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269028	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=8878 Win=122880 Len=0 TSval=3518264549 TSecr=155272227
7.269044	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=8878 Win=131072 Len=0 TSval=3518264549 TSecr=155272227
7.269819	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=8878 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=10326 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=11774 Ack=443 Win=65024 Len=1448 TSval=155272227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269875	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=13222 Win=126720 Len=0 TSval=3518264549 TSecr=155272227
7.269952	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=13222 Win=131072 Len=0 TSval=3518264550 TSecr=155272227
7.275377	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=13222 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275378	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=14670 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275379	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=16118 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=17566 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=19014 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275478	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=20462 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275479	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=21910 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275484	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=20462 Win=123776 Len=0 TSval=3518264556 TSecr=15527240
7.275509	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=23358 Win=120896 Len=0 TSval=3518264556 TSecr=15527240
7.275544	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=23358 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]

图 4.16: 发送 http 请求

4.3.5 服务器传输数据

在客户端发送 HTTP 请求之后，服务器开始返回 HTTP 响应数据。Wireshark 抓包分析显示，服务器通过 TCP 将 HTTP 响应分段发送给客户端，具体过程同第一次实验相同，主要是走 TCP 传输流程，这里展示一些特殊作用的包。

在**第一个响应包**中，我们可以看到这个包具有特殊的结构和标志位组合：



Time	Source	Destination	Protocol	Length	Info
7.075797	10.130.100.231	10.134.187.81	TCP	78	59310 → 9999 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3518264356 TSecr=0 SACK_PERM
7.086759	10.134.187.81	10.130.100.231	TCP	74	9999 → 59310 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=15527052 TSecr=3518264356
7.086878	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=3518264367 TSecr=15527052
7.263381	10.134.187.81	10.130.100.231	TCP	255	9999 → 59310 [PSH, ACK] Seq=1 Ack=443 Win=65024 Len=189 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268879	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=190 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=1638 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=3086 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268935	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=4534 Win=127232 Len=0 TSval=3518264549 TSecr=15527227
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=4534 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=5982 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269006	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=7430 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269028	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=8878 Win=122880 Len=0 TSval=3518264549 TSecr=15527227
7.269044	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=8878 Win=131072 Len=0 TSval=3518264549 TSecr=15527227
7.269819	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=8878 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=10326 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=11774 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269875	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=13222 Win=126720 Len=0 TSval=3518264549 TSecr=15527227
7.269952	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=13222 Win=131072 Len=0 TSval=3518264550 TSecr=15527227
7.275377	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=13222 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275378	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=14670 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275379	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=16118 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=17566 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=19014 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275478	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=20462 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275479	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=21910 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275484	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=20462 Win=123776 Len=0 TSval=3518264556 TSecr=15527240
7.275509	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=23358 Win=120896 Len=0 TSval=3518264556 TSecr=15527240
7.275544	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=23358 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]

图 4.17: 带响应信息的包

包结构分析 该数据包具有以下关键特征：

1. **三重功能组合**：此包同时承担三种功能：

- **流量控制**：通告窗口大小为 65,024 字节（缩放因子 256）
- **数据确认**：ACK 确认号 443，确认收到客户端全部 442 字节请求数据
- **数据传输**：PSH 标志推送 189 字节 HTTP 响应数据

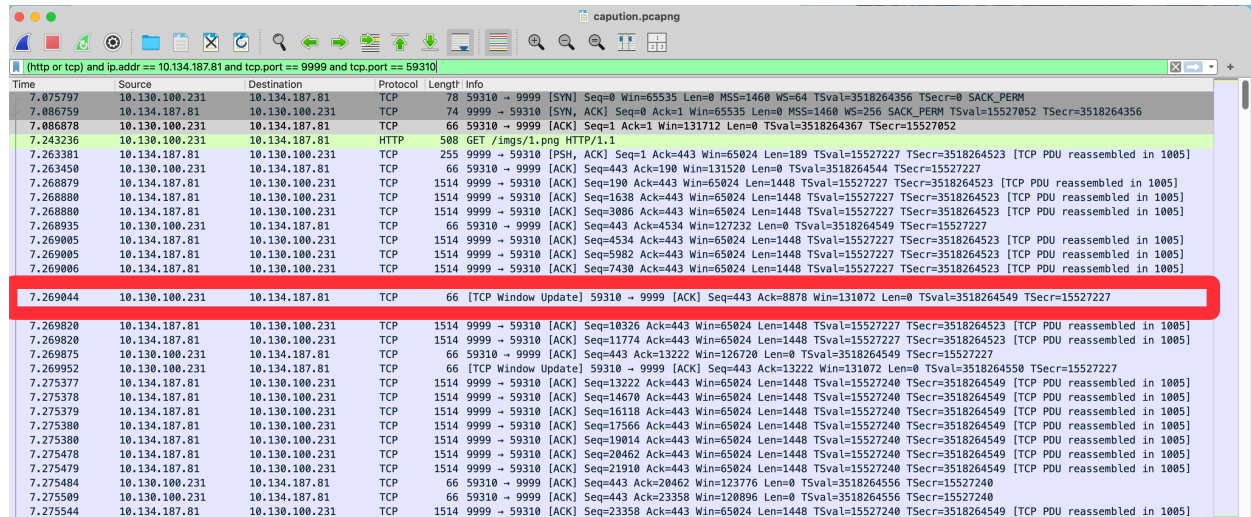
2. **序列号特征**：服务器序列号为 1，表明这是连接建立后服务器发送的第一个数据包。

3. **PSH 标志意义**：Push 标志要求接收方 TCP 层立即将数据传递给上层应用（HTTP），避免缓冲区延迟，这对于 Web 浏览器尽快开始页面渲染至关重要!!!

通信时序优化 此包体现了 TCP 协议的优化机制：**捎带确认**，将窗口更新、请求确认和响应数据传输合并到一个包中，提高了网络利用率。

HTTP 响应模式 基于 189 字节的数据长度和重组指示，可以猜测很有可能这里将 HTTP 的响应头部 header 先传了过来。

在 HTTP 响应数据的传输过程中，除了包含应用数据的 PSH 包外，还可以观察到专门用于流量控制的 TCP 窗口更新包，如图 4.18。这个包**仅包含 TCP 头部**，载荷长度为零（Len=0），专门用于流量控制而不携带应用数据。



Time	Source	Destination	Protocol	Length	Info
7.075797	10.130.100.231	10.134.187.81	TCP	78	59310 → 9999 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3518264356 TSecr=0 SACK_PERM
7.086759	10.134.187.81	10.130.100.231	TCP	74	9999 → 59310 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=15527052 TSecr=3518264356
7.086878	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=3518264367 TSecr=15527052
7.245236	10.130.100.231	10.134.187.81	HTTP	508	GET /imgs/1.png HTTP/1.1
7.263381	10.134.187.81	10.130.100.231	TCP	255	9999 → 59310 [PSH, ACK] Seq=1 Ack=443 Win=65024 Len=189 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.263450	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=190 Win=131520 Len=0 TSval=3518264544 TSecr=15527227
7.268879	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=190 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268880	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=1638 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.268935	10.130.100.231	10.134.187.81	TCP	1514	9999 → 59310 [ACK] Seq=3806 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269005	10.134.187.81	10.130.100.231	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=4534 Win=127232 Len=0 TSval=3518264549 TSecr=15527227
7.269005	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=5982 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269006	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=7430 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269044	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=8878 Win=131072 Len=0 TSval=3518264549 TSecr=15527227
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=10326 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=11774 Ack=443 Win=65024 Len=1448 TSval=15527227 TSecr=3518264523 [TCP PDU reassembled in 1005]
7.269875	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=13222 Win=126720 Len=0 TSval=3518264549 TSecr=15527227
7.269952	10.130.100.231	10.134.187.81	TCP	66	[TCP Window Update] 59310 → 9999 [ACK] Seq=443 Ack=13222 Win=131072 Len=0 TSval=3518264550 TSecr=15527227
7.275377	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=13222 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275378	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=14670 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275379	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=16118 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=17566 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275380	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=19014 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275478	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=20462 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275479	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=21910 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]
7.275484	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=20462 Win=123776 Len=0 TSval=3518264556 TSecr=15527240
7.275509	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=23358 Win=120896 Len=0 TSval=3518264556 TSecr=15527240
7.275544	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=23358 Ack=443 Win=65024 Len=1448 TSval=15527240 TSecr=3518264549 [TCP PDU reassembled in 1005]

图 4.18: 窗口更新包

4.3.6 发送 HTTP 响应

在客户端发送 HTTP 请求之后，服务器对请求进行处理，并通过已建立的 TCP 连接向客户端返回 HTTP 响应。从抓包结果可以看到，服务器返回的响应状态码为 200 OK，并携带所请求的 PNG 图片数据。由于响应数据量较大，HTTP 响应被拆分为多个 TCP 报文段进行传输，最终在客户端完成重组。

Time	Source	Destination	Protocol	Length	Info
7.376597	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=496854 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376598	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=498302 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376598	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=499750 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376598	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=501198 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376599	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=502646 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376599	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=504094 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376599	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=505542 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376600	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=506990 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376600	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=508438 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376601	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=509886 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376601	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=511334 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376601	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=512782 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376602	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=514230 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376602	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=515678 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376603	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=517126 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376603	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=518574 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376603	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=520022 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376604	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=521470 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376604	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=522918 Ack=443 Win=65024 Len=1448 TSval=15527330 TSecr=3518264631 [TCP PDU reassembled in 1005]
7.376750	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=524366 Win=262784 Len=0 TSval=3518264657 TSecr=15527330
7.376820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=524366 Ack=443 Win=65024 Len=1448 TSval=15527342 TSecr=3518264647 [TCP PDU reassembled in 1005]
7.376820	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=525814 Ack=443 Win=65024 Len=1448 TSval=15527342 TSecr=3518264647 [TCP PDU reassembled in 1005]
7.376821	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=527262 Ack=443 Win=65024 Len=1448 TSval=15527342 TSecr=3518264647 [TCP PDU reassembled in 1005]
7.376821	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=528710 Ack=443 Win=65024 Len=1448 TSval=15527342 TSecr=3518264647 [TCP PDU reassembled in 1005]
7.376871	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=530158 Win=262784 Len=0 TSval=3518264657 TSecr=15527342
7.376927	10.134.187.81	10.130.100.231	HTTP	822	HTTP/1.0 200 OK (PNG)
7.376927	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=530158 Ack=443 Win=65024 Len=1448 TSval=15527342 TSecr=3518264659 [TCP PDU reassembled in 1005]
7.376927	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [FIN, ACK] Seq=443 Ack=532363 Win=262784 Len=0 TSval=3518264659 TSecr=15527342
7.382919	10.134.187.81	10.130.100.231	TCP	66	9999 → 59310 [ACK] Seq=532363 Ack=444 Win=65024 Len=0 TSval=15527349 TSecr=3518264659

图 4.19: 发送最终响应

4.3.7 关闭 TCP 连接-连接释放

在 HTTP 响应数据传输完成之后，客户端与服务器通过 TCP 连接释放机制终止通信。根据抓包分析结果，如图 4.20 所示，可以观察到 TCP 连接关闭过程。

Time	Source	Destination	Protocol	Length	Info
7.376871	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=530158 Win=262784 Len=0 TSval=3518264657 TSecr=15527342
7.376876	10.134.187.81	10.130.100.231	TCP	1514	9999 → 59310 [ACK] Seq=530158 Ack=443 Win=65024 Len=1448 TSval=15527342 TSecr=3518264647 [TCP PDU reassembled in 1005]
7.376927	10.134.187.81	10.130.100.231	HTTP	822	HTTP/1.0 200 OK (PNG)
7.376963	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [ACK] Seq=443 Ack=532363 Win=260608 Len=0 TSval=3518264657 TSecr=15527342
7.376963	10.130.100.231	10.134.187.81	TCP	66	59310 → 9999 [FIN, ACK] Seq=443 Ack=532363 Win=262784 Len=0 TSval=3518264659 TSecr=15527342
7.382919	10.134.187.81	10.130.100.231	TCP	66	9999 → 59310 [ACK] Seq=532363 Ack=444 Win=65024 Len=0 TSval=15527349 TSecr=3518264659

图 4.20: 连接释放

1. **客户端发送 ACK（确认最后数据）** 在服务器发送完 HTTP 响应（HTTP/1.0 200 OK）后，客户端首先发送 ACK 报文，对服务器最后一个 TCP 数据段进行确认。该报文不携带数据，仅用于确认已成功接收全部响应内容，为连接释放做准备。
2. **客户端发送 FIN-ACK（主动关闭连接）** 随后，客户端向服务器发送 FIN, ACK 报文，表明客户端已经没有数据需要继续发送，请求关闭连接。此时客户端由 ESTABLISHED 状态进入 FIN_WAIT_1 状态，开始主动发起 TCP 连接释放过程。
3. **服务器发送 ACK（确认关闭请求）** 服务器收到客户端的 FIN 报文后，返回 ACK 报文进行确认，表示已知晓客户端的连接关闭请求。此时服务器进入 CLOSE_WAIT 状态，而客户端进入 FIN_WAIT_2 状态，连接进入半关闭阶段。

需要注意的是，在本次抓包中未观察到服务器发送单独的 FIN 报文。这是由于服务器在确认客户端 FIN 时，已无应用层数据需要继续发送，TCP 协议允许将关闭操作进行合并或直接完成连接释放。因此，抓包中表现为仅出现三个关键报文段，而非严格意义上的四次挥手。

总体来看，虽然连接释放过程中报文数量有所简化，但其本质仍符合 TCP 协议的连接释放机制，保证了连接在双方均完成数据传输后被安全、可靠地关闭。

5 总结心得

5.1 本次实验遇到问题 & 解决

- **本地回环**

由于本地回环会导致数据链路层被简化，这里无法分析这个部分的内容，最后借了同学电脑来进行实验抓包。

- **缓存响应**

抓包之前都会尝试过打开服务器网站，当我第一次抓包的时候发现并没有出现 200OK 的响应，而是出现 304 Not Modified，说明直接用了浏览器的缓存，而没有发送请求，最后关闭开发者工具中网络里面的启用缓存之后，刷新网站就不会有了。

5.2 总结

本次实验用 Wireshark 工具，实现了对 HTTP 交互过程的抓包，学习了如何在 Wireshark 中使用条件过滤。通过对抓包信息的分析，详细学习了 HTTP 的交互过程，了解了**建立 TCP 连接 → 发送 HTTP 请求 → 服务器传输数据 -> 发送 HTTP 响应 → 关闭 TCP 连接**的完整交互过程。